

WIS2 Champions: “cache” session

(Jeremy Tandy, MAR-2026)



WORLD
METEOROLOGICAL
ORGANIZATION



- 1/ Global Caches**
- 2/ Sensor Centres for Global Caches**
- 3/ Global Replay (GRep) service**



WORLD
METEOROLOGICAL
ORGANIZATION



1/ **Global Caches**

1.A/ Tech Regs: Manual on WIS, Volume II



WORLD
METEOROLOGICAL
ORGANIZATION





Terms and definitions

Global Cache. A Global Service that provides highly available access to WIS data via the Internet.

Part 1: Organisation and responsibilities

1.1 Principles of WIS 2.0

1.1.2 WIS 2.0 is designed in accordance with the following principles:

(7) WIS 2.0 requires all services that provide real-time distribution of messages (containing data or notifications about data availability) to cache/store the messages for a minimum of 24 hours and allow users to request cached messages for download;

1.4 Interaction and collaboration among WIS Centres

1.4.4 A WIS centre operating a Global Cache shall provide access to locally stored copies of core data for real-time or near-real-time exchange and discovery metadata published by all NCs and DCPCs.



Part IV. WIS Technical Specifications

4.5 WIS-TECHSPEC-4: OPERATING A GLOBAL CACHE

4.5.1 A Global Cache shall operate as a highly available storage and download service for:

(i) Core data, as defined in Resolution 1 (Cg-Ext(2021)), where programme requirements require sharing in real time or near real time; and (ii) Discovery metadata records.

4.5.2 A Global Cache shall download core data and discovery metadata from WIS nodes and other Global Caches to provide for reliable, low-latency access to those resources via WIS.

4.5.3 A Global Cache shall subscribe to at least one Global Broker for notifications concerning core data and discovery metadata, as specified in the *Guide to WIS*, Volume II.

4.5.4 Based on its received notifications, a Global Cache shall download core data from WIS nodes or other Global Caches and store them for a duration compatible with the real-time or near-real-time schedule of the data and not less than 24 hours.

4.5.5 Based on its received notifications, a Global Cache shall download discovery metadata records from WIS nodes or other Global Caches and store them for a minimum duration of 24 hours.

4.5.6 Data and discovery metadata available for download from a Global Cache shall be accessible via a URL using at least one of the protocols specified in the *Guide to WIS*, Volume II.

4.5.7 A Global Cache shall publish notifications to a Message Broker indicating the availability of data and discovery metadata resources. Notifications shall include the URL for downloading resources from the Global Cache and shall use the format and protocol specified in the *Guide to WIS*, Volume II.

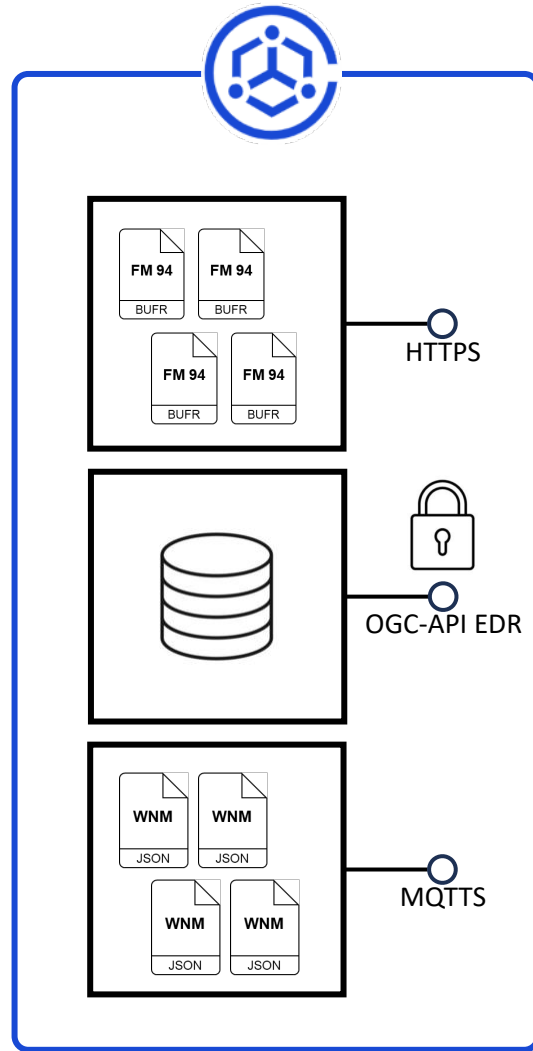
1/ Global Caches

1.B/ Where do Global Caches fit in the WIS2 ecosystem?



WORLD
METEOROLOGICAL
ORGANIZATION





Global Broker: Provides highly available global distribution of notifications about new (meta)data available for download from WIS2 Nodes or Global Caches



Global Cache: Provides highly available HTTP(S) download of Core¹ data cached from WIS2 Nodes



Saudi GC not shown.



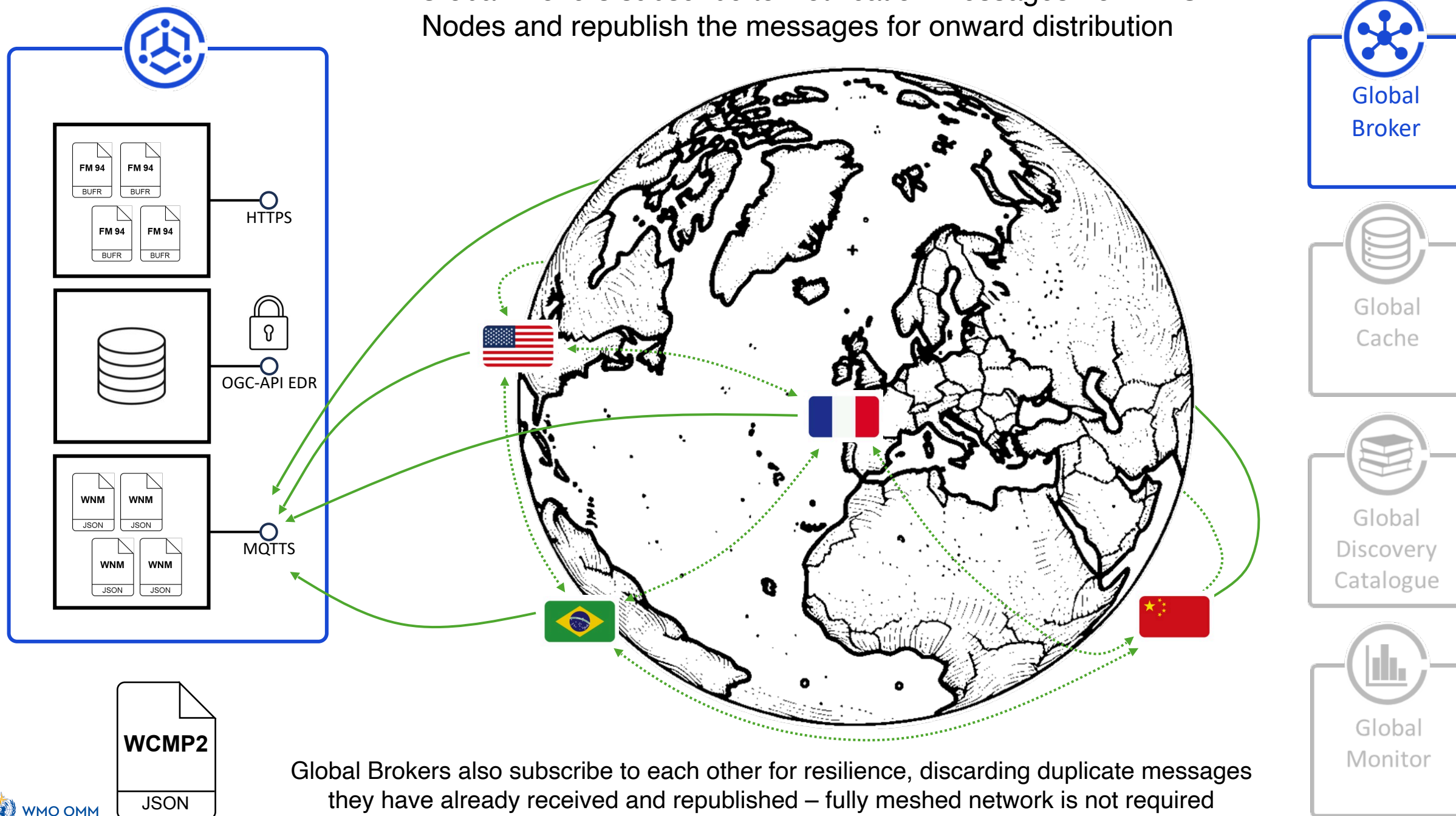
Global Discovery Catalogue: Provides users an API to discover datasets and services



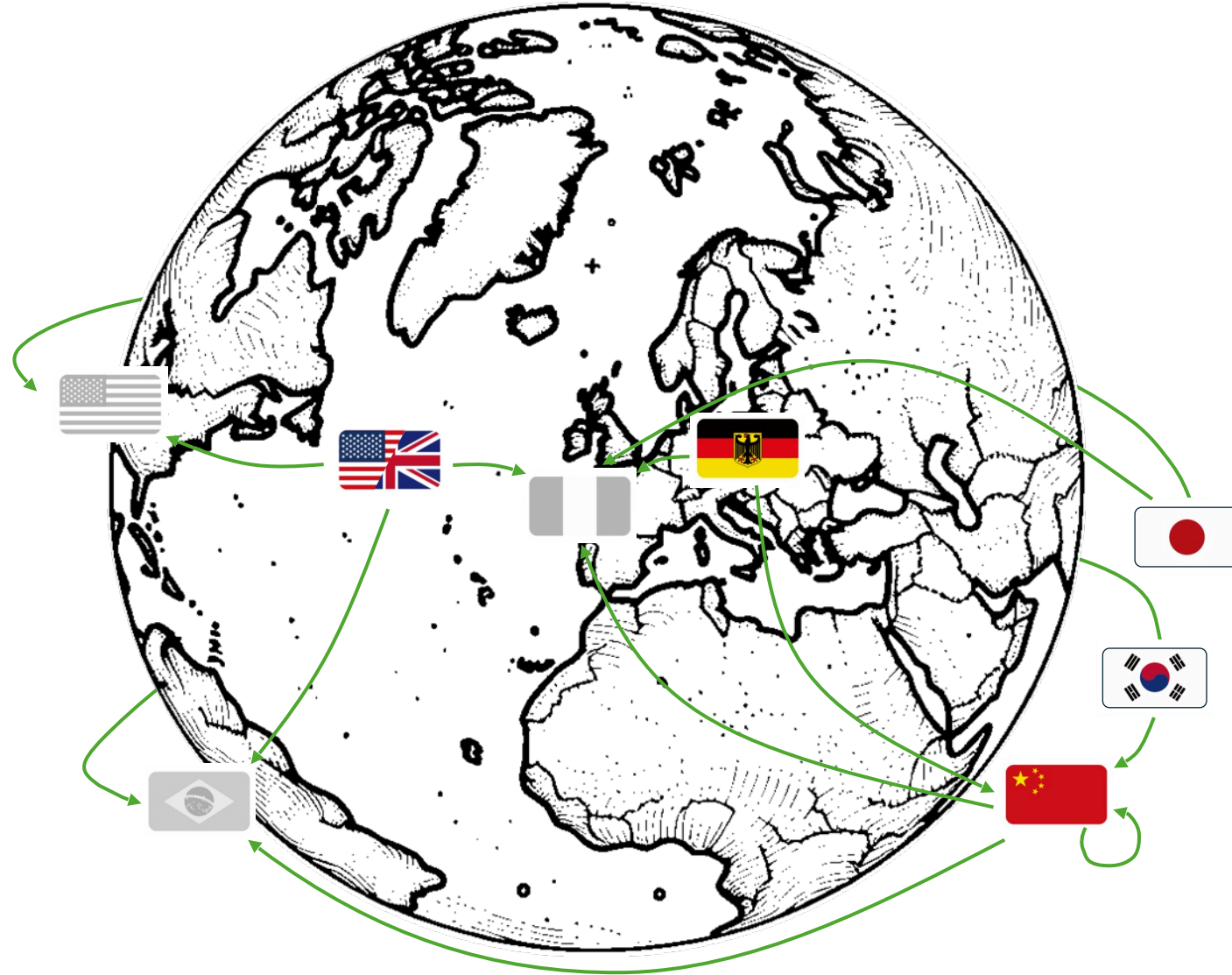
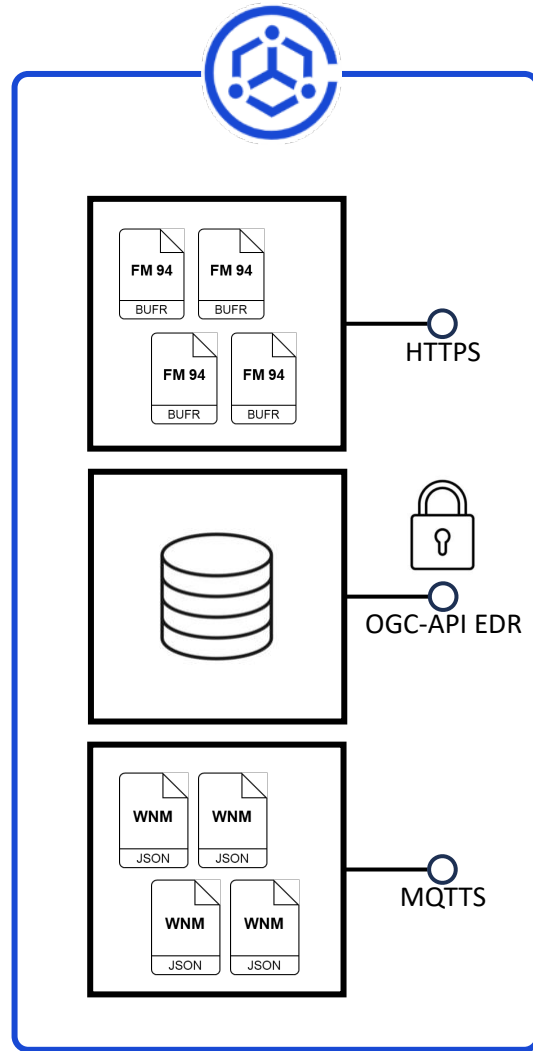
Global Monitor: Reports the health of WIS2 using metrics aggregated from all Global Services



4x Global Brokers subscribe to Notification Messages from WIS2 Nodes and republish the messages for onward distribution

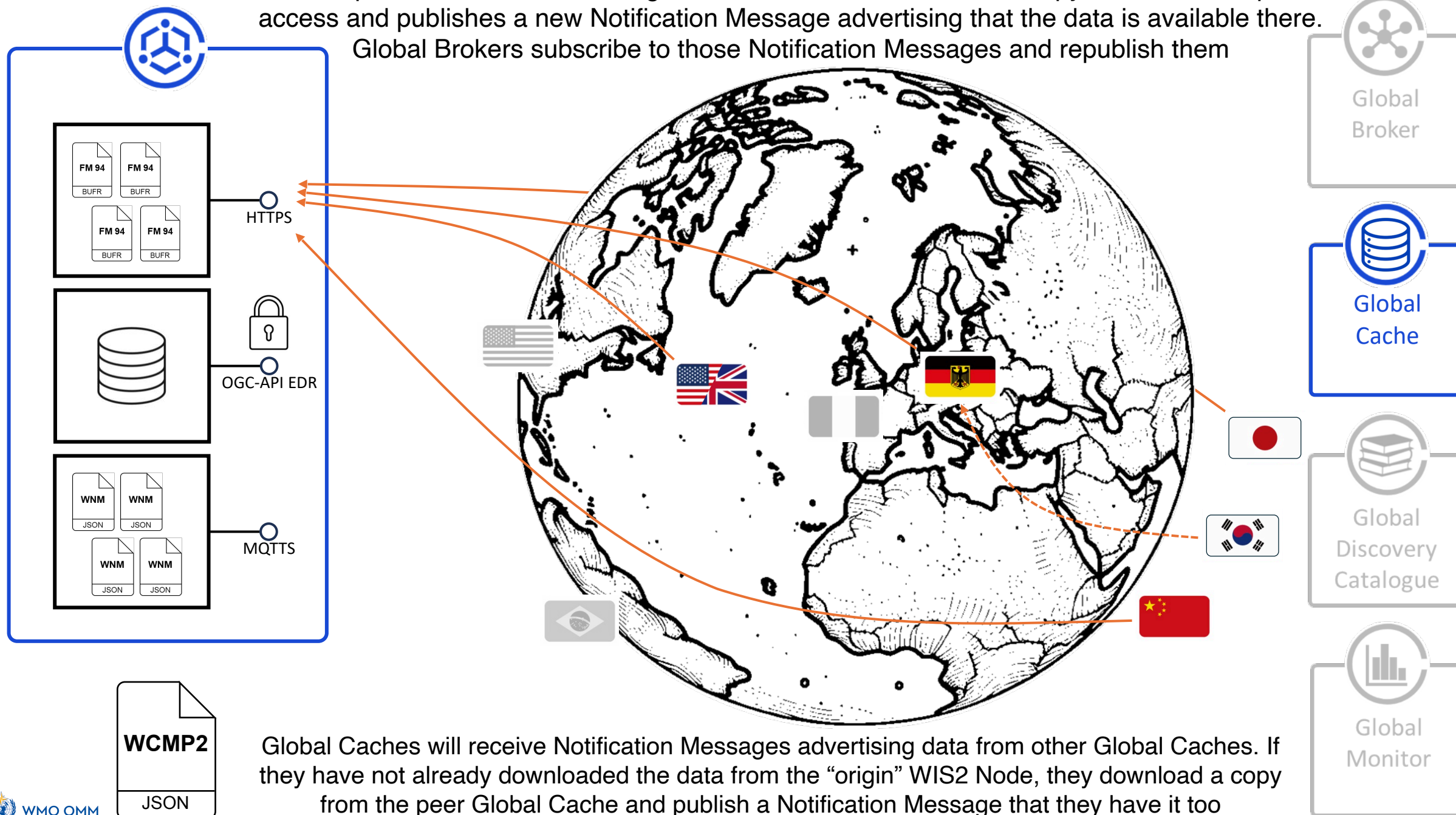


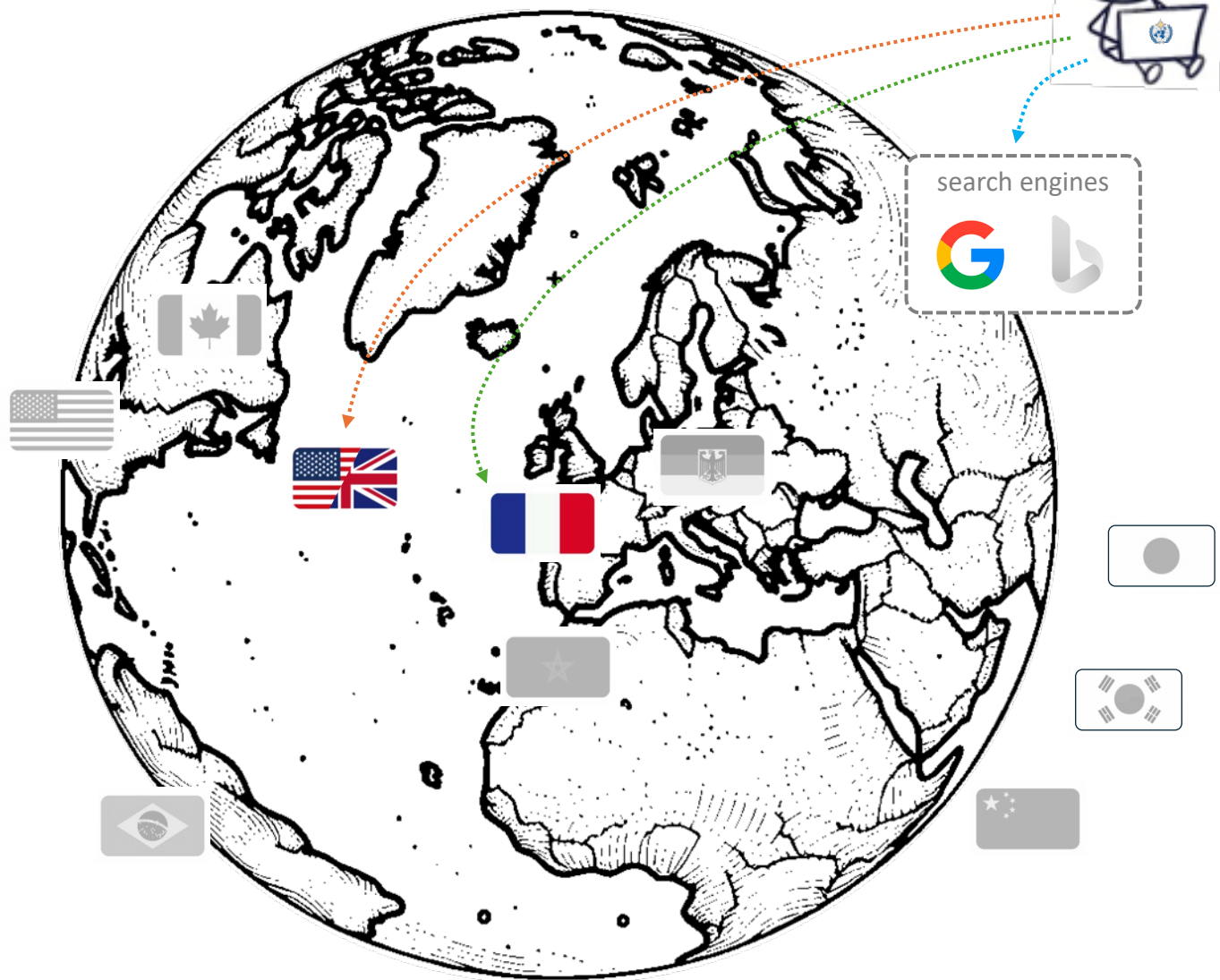
6x Global Caches each subscribe to Notification Messages from at least 2x Global Brokers, discarding duplicate messages



(Saudi GC not shown)

On receipt of a Notification Message, a Global Cache downloads a copy of the data for public access and publishes a new Notification Message advertising that the data is available there. Global Brokers subscribe to those Notification Messages and republish them





“ So – I start by searching for the data that I need using my favourite search engine or a Global Discovery Catalogue

Then subscribe to notifications from one of the Global Brokers listed in the metadata

And to get the data I wait for the first Notification Message to arrive which will point me to where the data is ...

If it's **Core** data, I download from the Global Cache mentioned in the Notification Message

If it's **Recommended** data, I download from the origin WIS2 Node

(And for non-real-time data, the discovery metadata points me to the WIS2 Node so I can access data services there)

1/ Global Caches

1.C/ Tech Regs: Guide to WIS, Volume II (updated for FT2026-1)



WORLD
METEOROLOGICAL
ORGANIZATION





Technical considerations (Guide, part 2 - updated for FT2026-1)

2.7.4 Global Cache: [Summary] Global Caches act as transparent intermediaries that serve cached copies of WMO core data and republish notification messages that advertise the availability of cached data. GCs shield data providers from direct consumer access while ensuring consumers always receive a consistent notification stream regardless of whether data is cached.

2.7.4.1 Technical Considerations: [Summary]

- 3 components: data server, MQTT message broker (& client), cache management component
- Subscribe to topics `origin/a/wis2/#` and `cache/a/wis2/#` (origin and cache)
- Cached content: (near) real-time Core data and WCMP2 records; minimum of 24-hours; new and updated (meta)data objects, plus the daily archive of WCMP2 from Global Discovery Catalogs
- (Meta)data objects identified by WNM properties `data_id` and `pubtime`; duplicate resources or out-of-sequence updates are discarded
- Objects with WNM property `cache` set to **false** are not cached; WNM republished on `cache/` topic
- GC may decide not to cache an object; e.g., if the object is considered too large; treat as though property `cache` is set to **false**
- Attempt to download from the `canonical` (or `updated`) link in WNM
- Download objects in parallel to ensure that downloads are not blocking each other
- Objects (encoded size < 4096b) **may** be embedded in `content` property of WMN; “in-line” data



Technical considerations (Guide, part 2 - updated for FT2026-1)

2.7.4.1 Technical Considerations (cont.):

- If WNM includes `integrity` property validate object integrity
- Publish copy of object on GCs HTTP data server - even for in-line data
- Publish new WNM advertising availability of cached object on `cache/` topic: new message `id`, add property `global-cache` with value set to GCs centre-id, link pointing to GCs HTTP data server
- If object download or integrity check fails, attempt to cache from alternate source (e.g., triggered by arrival of new WNM for the object from another GC)
- Multiple GCs for resilience; approximate match of content (maybe small differences); no formal synchronisation
- The multiplicity of GCs is transparent to data consumers; determining which GC (e.g., as a preferred source) can be done via the `global-cache` property and link URL of a WNM
- GC may provide a browsable interface on their cache; subscribing to WNM and downloading via the link URL is the primary access mechanism
- GC should operate with a fixed IP address so that WIS2 Nodes can permit access to download resources based on IP address filtering; GC should also operate with a publicly resolvable DNS name pointing to that IP address
- GC publishes metrics every 60-seconds
- As a convention, the Global Cache centre-id will be `{tld}-{centre-name}-global-cache`

1/ Global Caches

1.D/ Metrics



WORLD
METEOROLOGICAL
ORGANIZATION





Global Cache metrics [gc.csv](#)

Metrics for Global Services are intended *only* to assess if WIS2 is properly functioning - not supporting analytics.

[README](#) explains how the metrics are structured and the meaning of specific labels

- `centre_id`: indicates whose status this metric refers to. For Global Caches: use the centre ID of the WIS2 component from where a resource is downloaded and cached, e.g., the origin WIS2 Node (parsed from the 4th field of the WIS2 Topic) or a Global Cache (parsed from the `properties.global-cache` object in the WIS2 Notification Message).
- `report_by`: the `centre-id` of the WIS2 component reporting this metric.

Name	Labels	Description
<code>wmo_wis2_gc_downloaded_total</code>	<code>centre_id report_by</code>	Number of data items downloaded to Global Cache: <i>This metric is incremented by 1 each time the Global Cache successfully downloads and validates a new resource or an update to a resource published more recently than any previously received messages for that resource. This metric is not incremented for duplicate resources or out-of-sequence updates.</i>
<code>wmo_wis2_gc_downloaded_errors_total</code>	<code>centre_id report_by</code>	Number of download errors for Global Cache data: <i>This metric is incremented by 1 each time the Global Cache fails to download a resource that it is attempting to cache.</i>
<code>wmo_wis2_gc_last_download_timestamp_seconds</code>	<code>centre_id report_by</code>	Timestamp of last successful download: <i>This metric is set to the current time (timestamp - the number of seconds since Unix epoch) each time the Global Cache successfully downloads a new or updated resource.</i>
<code>wmo_wis2_gc_integrity_failed_total</code>	<code>centre_id report_by</code>	Number of messages for which the integrity check failed: <i>This metric is incremented by 1 each time the Global Caches fails to validate the integrity of a new or updated resource that it is attempting to cache.</i>

1/ Global Caches

1.E/ Test scripts



WORLD
METEOROLOGICAL
ORGANIZATION





Global Cache Service Testing

Test scripts: a great way to understand how the Global Services should function

Preamble:

- [WIS2DEV.io](https://wis2dev.io) test environment
- Rémy's fake WIS2Nodes and other utilities

Not included in the GC tests:

- Checking that Global Caches ignore Recommended data.
- Setting `properties.global-cache` in WNMs published by the Global Cache. [FT2025-2]
- Monitoring Event Message publication. [INFCOM-4]
- Extracting embedded ("in-line") data from WIS2 Notification Messages.



Global Cache Service Testing

Simplified test scripts: wmo-im/wis2-champions/Bali-workshop-MAR2026/GC_TESTS.md

1. Functional Tests

1. MQTT Broker Connectivity
2. GC MQTT Broker Subscription
3. WIS2 Notification Message (WNM) Processing
4. Cache False Directive
5. Source Download Failure
6. Data Integrity Check Failure
7. WIS2 Notification Message Deduplication
8. WIS2 Notification Message Deduplication (Alternative 1)
9. WIS2 Notification Message Deduplication (Alternative 2)
10. Data Update

2. Performance Tests

1. WIS2 Notification Processing Rate
2. Concurrent Client Downloads



Global Cache Service Testing

Sidebar: embedded (“in-line”) data

WNM spec: The `content` property allows for the inclusion of data in the notification message when the length of the data, once encoded, is smaller than 4 096 bytes.

For embedded data, the WIS2-to-GTS Gateway tests are illustrative. Summary:

1. Inline data (data embedded within the WNM) retrieved OK
2. Failure to retrieve inline data, success to retrieve data from URL
3. Failure to retrieve inline data, failure to retrieve data from URL

Discussion: what *is* the correct behaviour for this in-line data edge case?

- A. Inline data OK - but the HTTP resource is unreachable (e.g., it's not on the server, or the server is offline). Currently this wouldn't trigger an error - because the GCs never attempt the download. If we want to change this, we could force the GCs to do a HTTP HEAD request to see if the resource/server are available but not actually download the data. But this would be a change to the GC behaviour.
- B. Inline data not OK and there is no URL provided (this is pretty small edge case). If the GC fails to extract embedded data it should attempt to download from the URL. If that also fails, we increment `wmo_wis2_gc_download_errors_total`. If there's no URL to try to download from, perhaps the lack of a URL should cause the `wmo_wis2_gc_download_errors_total` metric to be incremented too?

1/ Global Caches

1.F/ Implementation



WORLD
METEOROLOGICAL
ORGANIZATION





6 Global Caches

1. **China:** cn-cma-global-cache
 - BROKER=<mqtt://gc.wis.cma.cn:8883>
 - USERNAME=everyone
 - PASSWORD=Everyone@100081&
2. **UK/USA:** data-metoffice-noaa-global-cache
 - BROKER=<mqtt://wis2cache.globaldata.nws.noaa.gov:8883>
 - USERNAME=everyone
 - PASSWORD=everyone
3. **Germany:** de-dwd-global-cache
 - BROKER=<mqtt://wis2.dwd.de:8883>
 - USERNAME=everyone
 - PASSWORD=everyone
4. **Japan:** jp-jma-global-cache
 - BROKER=<mqtt://mqtt.wis-jma.go.jp>
 - USERNAME=france
 - PASSWORD=juz*ai~v0eiNieT8bi
5. **South Korea:** kr-kma-global-cache
 - BROKER=<mqtt://wis2gc.kma.go.kr:8883>
 - USERNAME=everyone
 - PASSWORD=everyone
6. **Saudi Arabia:** sa-ncm-global-cache
 - BROKER=<mqtt://wis2.ncm.gov.sa:8883>
 - USERNAME=everyone
 - PASSWORD=everyone



6 Global Caches: deep dive on data-metoffice-noaa-global-cache

1. China: cn-cma-global-cache

- BROKER=<mqtt://gc.wis.cma.cn:8883>
- USERNAME=everyone
- PASSWORD=Everyone@100081&

2. UK/USA: data-metoffice-noaa-global-cache

- BROKER=<mqtt://wis2cache.globaldata.nws.noaa.gov:8883>
- USERNAME=everyone
- PASSWORD=everyone

3. Germany: de-dwd-global-cache

- BROKER=<mqtt://wis2.dwd.de:8883>
- USERNAME=everyone
- PASSWORD=everyone

4. Japan: jp-jma-global-cache

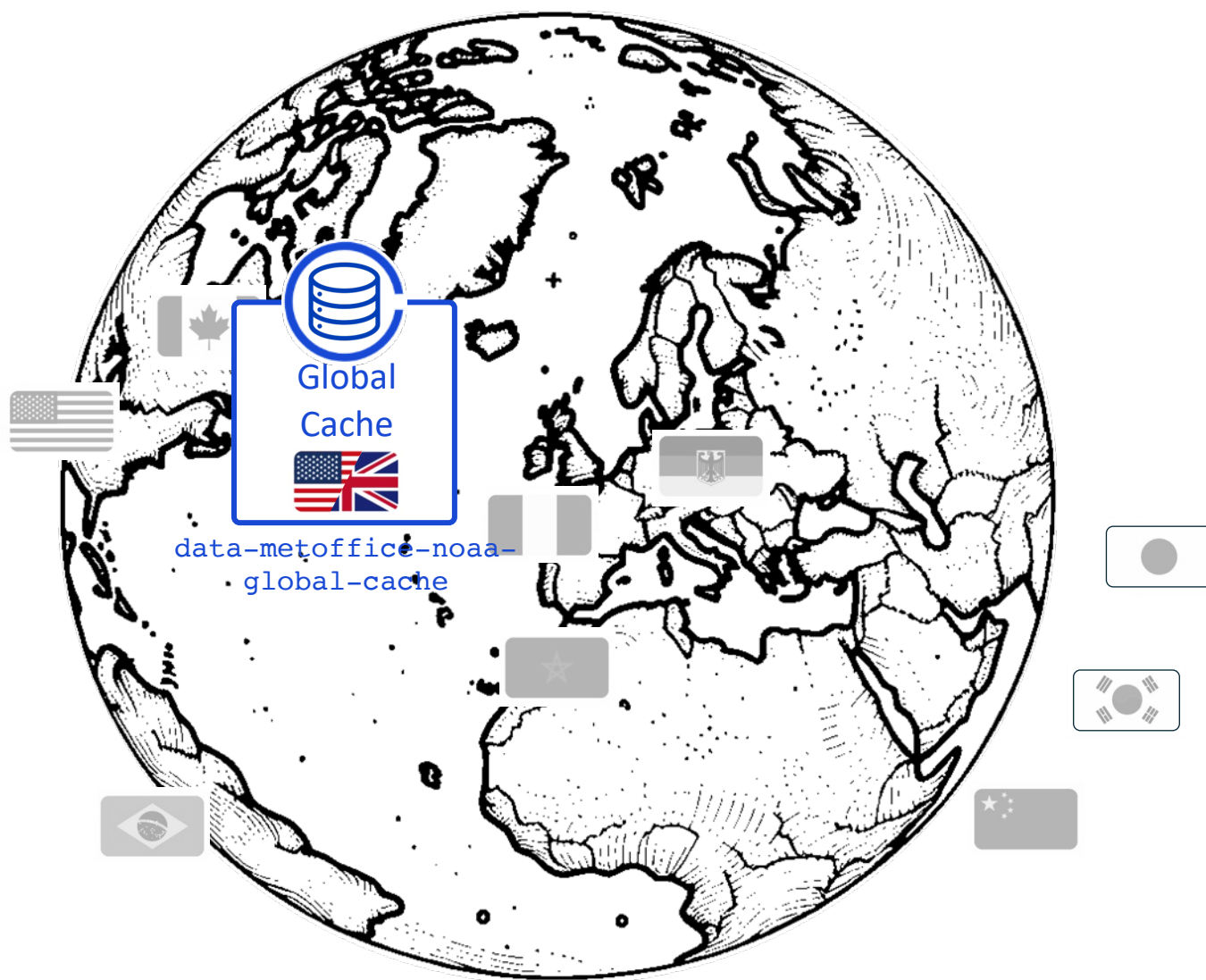
- BROKER=<mqtt://mqtt.wis-jma.go.jp>
- USERNAME=france
- PASSWORD=juz*ai~v0eiNieT8bi

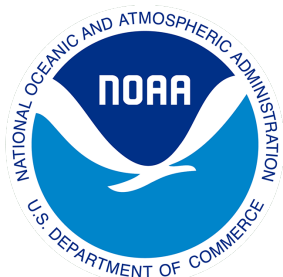
5. South Korea: kr-kma-global-cache

- BROKER=<mqtt://wis2gc.kma.go.kr:8883>
- USERNAME=everyone
- PASSWORD=everyone

6. Saudi Arabia: sa-ncm-global-cache

- BROKER=<mqtt://wis2.ncm.gov.sa:8883>
- USERNAME=everyone
- PASSWORD=everyone



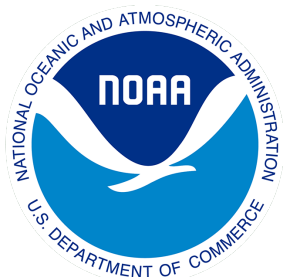


data-metoffice-noaa-global-cache

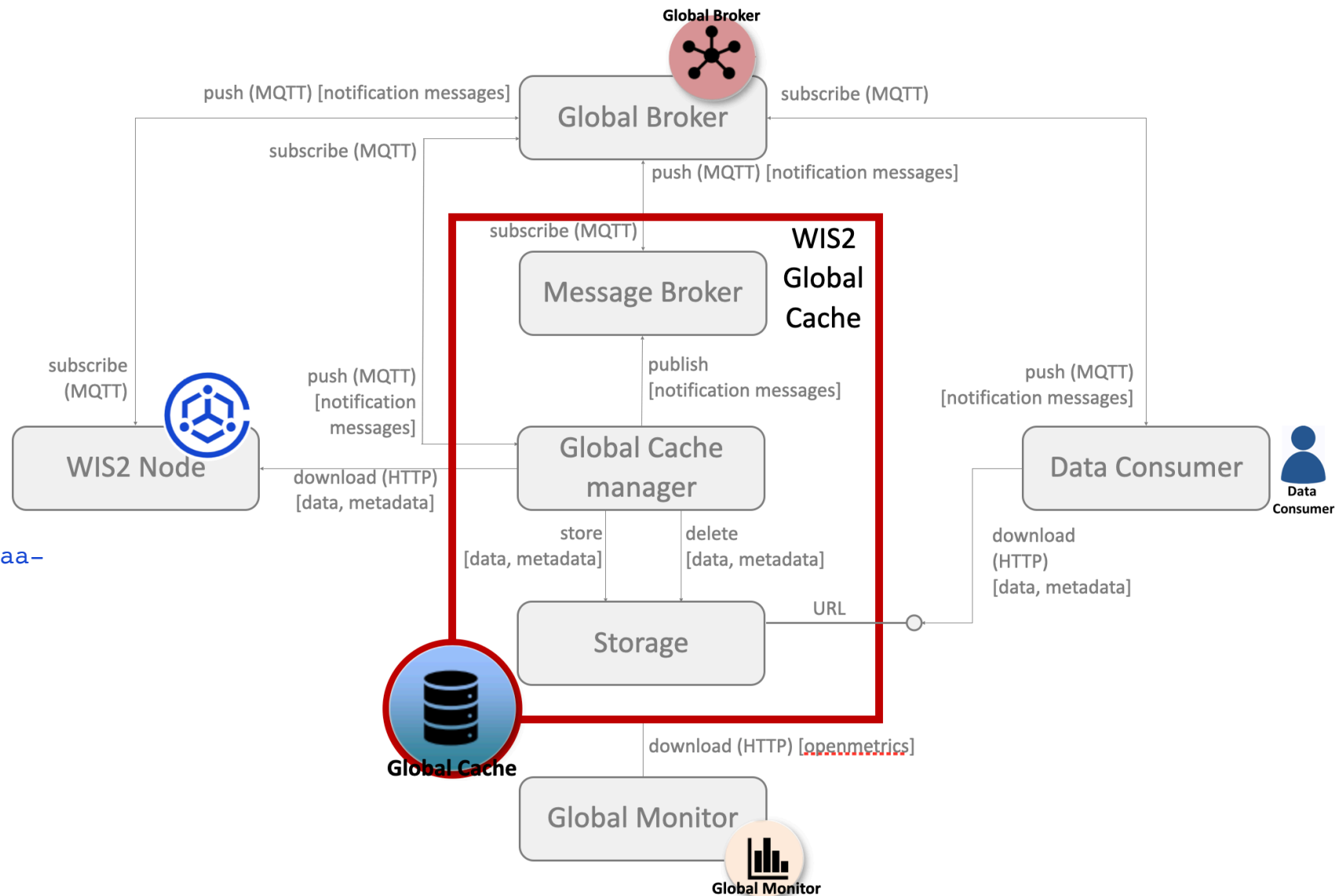
data-metoffice-noaa-global-cache:

- Implemented by Synoptic (@jmaxmarno)
- <https://github.com/synoptic/wis2-global-cache> [private]
- Not a reference implementation; also used in the [WIS2dev.io](https://wis2dev.io) test environment.
- The service is deployed using AWS CDK, ensuring a scalable and efficient caching infrastructure.
- Data hosted on Amazon Open Data Registry <https://registry.opendata.aws/wis2-global-cache/> [Registry card], <https://wis2globalcache.s3.amazonaws.com/index.html> [S3 bucket]
- Service documentation: <https://registry.opendata.aws/wis2-global-cache/>

Global Cache content: (near) real-time **Core** data¹ | 24-hour retention

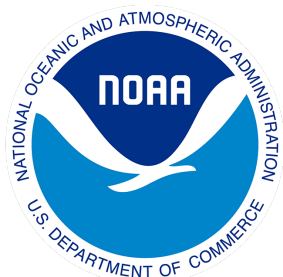


data-metoffice-noaa-
global-cache



Global Cache content: (near) real-time **Core** data¹ | 24-hour retention

¹ WMO data sharing covers 6 Earth system domains; observations, predictions, re-analyses, etc. – see the [WMO Unified Data Policy \(Resolution 1\)](#) for more information



The Amazon
Sustainability
Data Initiative

ASDI



data-metoffice-noaa-
global-cache

Cloud Development Kit:
IaaS deployment

Lambda: Serverless processing of
messages grouped into batches;
retrieving data for caching, etc.

SQS: Event driven architecture

ElastiCache (Redis): Durable state
management for message de-duplication

S3: persistent object storage for cache data
s3::wis2-global-cache

CloudFront: content delivery network for
low-latency global data access

Global Cache Manager



Fargate: Container management for custom apps

Paho: MQTT client that subscribes to Global Brokers

EMQX: MQTT broker for publishing notification
messages about cached data

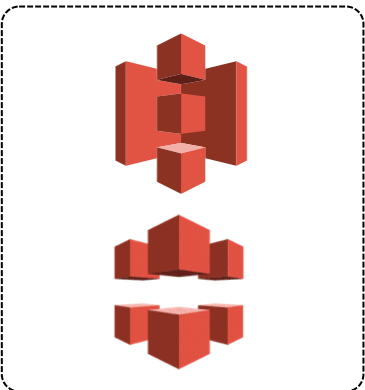


ASDI

Global Cache content: (near) real-time **Core** data¹ | 24-hour retention

¹ WMO data sharing covers 6 Earth system domains; observations, predictions, re-analyses, etc. – see the [WMO Unified Data Policy \(Resolution 1\)](#) for more information

Global Cache Manager

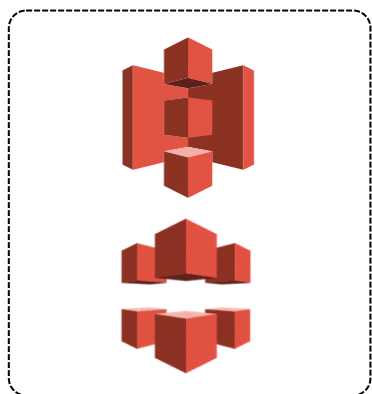


ASDI

Components:

1. MQTT Client Service (paho MQTT - Fargate): Uses a Fargate cluster to deploy one mqtt listener client per global broker (BR, FR, and US). Global Broker connection strings are stored in Secrets Manager. The client listens for incoming messages on the appropriate topics using paho MQTT and then queues the messages on SQS.
Stack File: `deploy/stacks/wis2_client_stack.py`
2. Queue (SQS): Deploys an SQS queue to handle message queuing. It ensures reliable message delivery and processing. The queue service includes a dead-letter queue for failed messages and a queue policy for access control.
Stack File: `deploy/stacks/wis2_queue_stack.py`
3. Lambda Manager (python Lambda): The Manager Lambda is responsible for processing messages from the queue. It reads messages from the queue, processes them, stores them in the cache, and publishes the cache messages if successful.
4. Metrics and duplicate detection (redis elasticache): The cache is implemented using redis Elasticache replication enabled cluster. The cache is used to store message data_ids, and their datetimes and to detect duplicate WIS2 notification messages. The cache is also used to store metrics data, such as the number of messages processed, the number of failures, and the datetime of the last successfully downloaded message that are returned by the metrics lambda.
5. MQTT Broker (EMQX - Fargate): The local MQTT broker for the GC service. The broker is used to publish cache messages to subscribed clients (other WIS2 global services). An admin dashboard is also available to monitor the broker.
6. Metrics Lambda (python Lambda): The Metrics Lambda is responsible for returning metrics data from the redis cache. It reads metrics data from the cache and returns the data to the client. This is coordinated by the API Gateway.

Global Cache Manager



ASDI

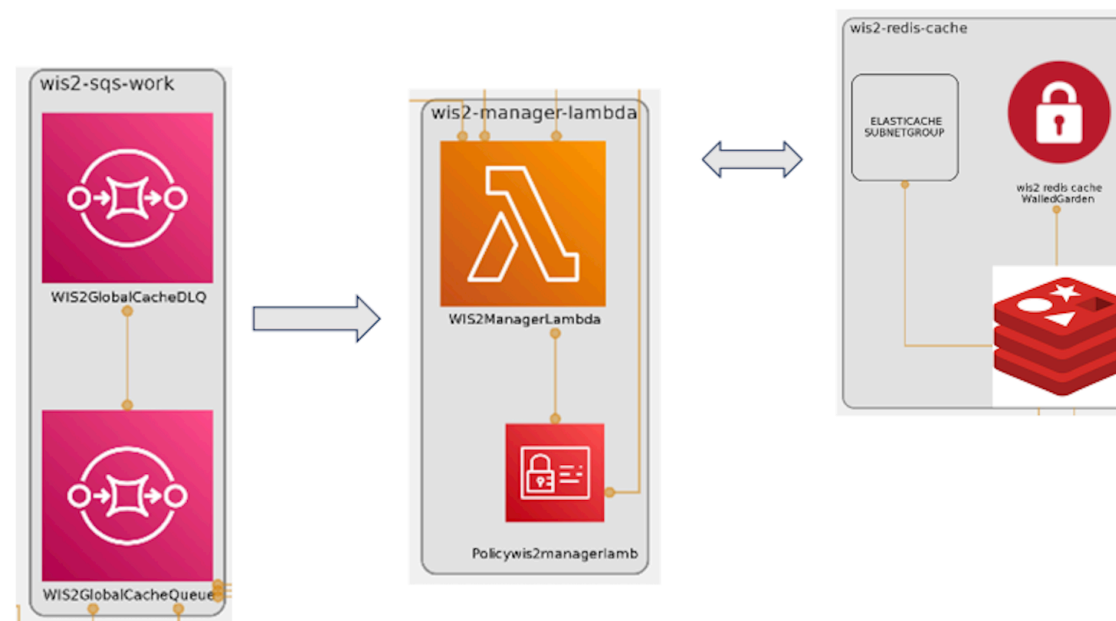
Lambda manager (the “heart” of the Cache Manager component):

This Lambda sits in the WIS2 global cache pipeline. It consumes batches of data-availability notification messages from SQS, deduplicates them via Redis, downloads and archives the actual data to S3 when needed, then republishes the notification to downstream MQTT subscribers. Errors at any stage are caught, logged, and themselves published to a dedicated MQTT error topic so they remain observable.

LAMBDA MANAGER — PSEUDOCODE

Message Processing

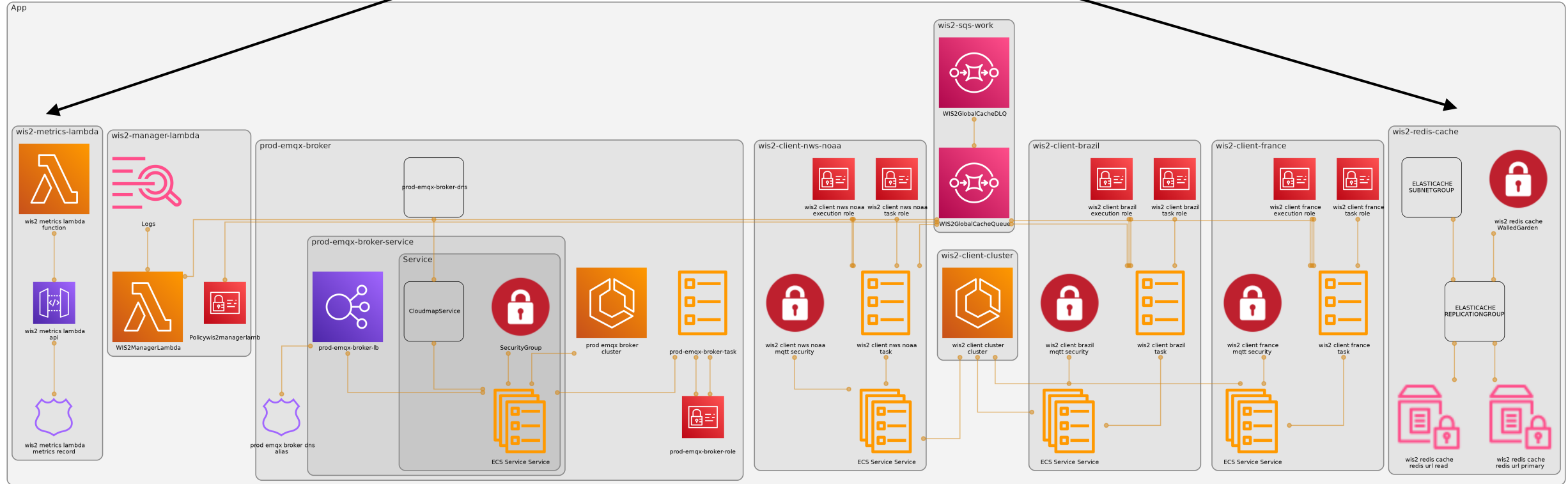
- AWS Lambda triggered by a Queue (AWS Simple Queue Service)
- ElastiCache (Redis) for deduplication
- Asynchronous processing messages in batches
- Message uniqueness (**data_id+pubtime**)
 - 'update' not currently enabled



Full component diagram

Additionally, see metrics implementation; metrics are available in realtime, no reset/rotations

3 components for metrics: AWS Lambda + API Gateway + Redis Cache



1/ Global Caches

1.G/ Tutorials



WORLD
METEOROLOGICAL
ORGANIZATION



2/ Sensor Centre for Global Cache

2.A/ Tech Regs: Guide to WIS, Volume II (updated for INFCOM-4)



WORLD
METEOROLOGICAL
ORGANIZATION



Sensor Centres for Global Caches (Guide, part 3 - updated for INFCOM-4)

3.3.3 Sensor Centres for Global Caches (SCGC)

The aim of the SCGC is to compare the behaviour of Global Cache(s)

In WIS 2.0 each Global Cache is independent of the other Global Caches. According to the specification of WIS 2.0, Global Cache (see [https://wmo-im.github.io/wis2-guide/guide/wis2-guide-APPROVED.html# 2 7 4 1 technical considerations](https://wmo-im.github.io/wis2-guide/guide/wis2-guide-APPROVED.html#2741technicalconsiderations)) :

Global Caches will operate independently of one another. Each Global Cache will hold a full copy of the cache – although there may be small differences between the various Global Caches as data availability notification messages propagate through WIS to each one. There is no formal synchronization between Global Caches.

[Summary] What can we measure? What information helps to identify and resolve GC problems or detect anomalies from WIS2 node (e.g., too frequent reuse of same `data_id`)? How can we objectively measure performance of GCs - key performance indicators (KPI)?

1. Average delay to cache data published by WIS2 Node?
2. Are all Core data objects published (with property `cache` set to **true**) by WIS2 Nodes are available in GCs?
3. Are objects missed by some or (more critically) all GCs?

Convention: SCGC centre-id will be `{tld}-{centre-name}-sensor-centre-global-cache`.

Sensor Centres for Global Caches (Guide, part 3 - updated for INFCOM-4)

3.3.3.1 Technical Considerations [Summary]

The processing of this Sensor Centre is as follows:

- Subscribe to `origin/a/wis2/...` on at least one Global Broker
- Subscribe to `cache/a/wis2/...` on at least one Global Cache
- Subscription on `cache/` must be done on the broker of the Global Cache
- Topics on `origin/` and `cache/` must match - it could be `#` or a particular centre-id on at least one Global Cache.

On the subscription to the Global Broker(s):

- Discard messages for Recommended data - these are not be cached
- Detect any duplicates `data_id` not including a `rel: update` within a period of at least 1 hour
- Store the time that the message is received

On the subscription to the Global Cache(s):

- For each data object (`data_id + pubtime`) calculate difference in time between arriving at origin and cache
- Update the `wmo_wis2_scgc_cache_delay_seconds` metric with this value
- Bucket the delay into threshold groups (e.g., less than 120s, less than 300s, less than 600s); increase by 1 the `wmo_wis2_scgc_messages_cached_delay_total` using the threshold as a label
- If no message for the `data_id` is received at cache after the highest threshold (here 600s), increase by 1 `wmo_wis2_scgc_messages_missed_total`
- Compare the `canonical` (or `update`) link in origin and cache messages; if identical, the GC has not cached the object, increase by 1 `wmo_wis2_scgc_messages_cache_override_total`

If no Global Cache has cached the data, increase by 1 `wmo_wis2_scgc_messages_missed_all_total`

2/ Sensor Centre for Global Cache

2.B/ Metrics



WORLD
METEOROLOGICAL
ORGANIZATION



Sensor Centre Global Cache metrics [scgc.csv](#)

Name	Labels	Description
wmo_wis2_scgc_cache_delay_seconds	global_cache centre_id report_by	Delay between origin and cache message
wmo_wis2_scgc_messages_cached_total	global_cache centre_id report_by	Number of data files cached for centre_id
wmo_wis2_scgc_messages_cached_delay_total	global_cache centre_id delay_threshold report_by	Number of data files cached for centre_id within defined delay threshold (120s 300s 600s)
wmo_wis2_scgc_messages_duplicates_total	centre_id report_by	Number of messages with identical data_id and no rel=update
wmo_wis2_scgc_messages_published_total	centre_id report_by	Number of cacheable data files published
wmo_wis2_scgc_messages_missed_total	global_cache centre_id report_by	Number of cacheable data not in global cache
wmo_wis2_scgc_messages_missed_all_total	centre_id report_by	Number of cacheable data not in any cache
wmo_wis2_scgc_messages_cache_override_total	global_cache centre_id report_by	Number of cacheable data with override cache directive

- Cache “delay” is grouped into buckets. The spec says 0-120s, 120-300s, and 300s-600s.
- In current (local) SCGC implementation 0-120s, 120-600s, and 600s-1200s.
- “Missed” is when the delay exceeds the bucket duration (CACHE_DELAY_MAX); 600s in the spec, and currently set to 1200s (20 minutes) in the SGGC implementation.

2/ Sensor Centre for Global Cache

2.C/ Implementation



WORLD
METEOROLOGICAL
ORGANIZATION

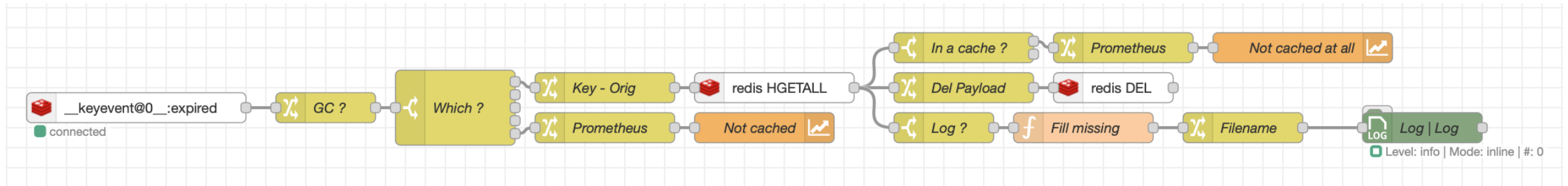


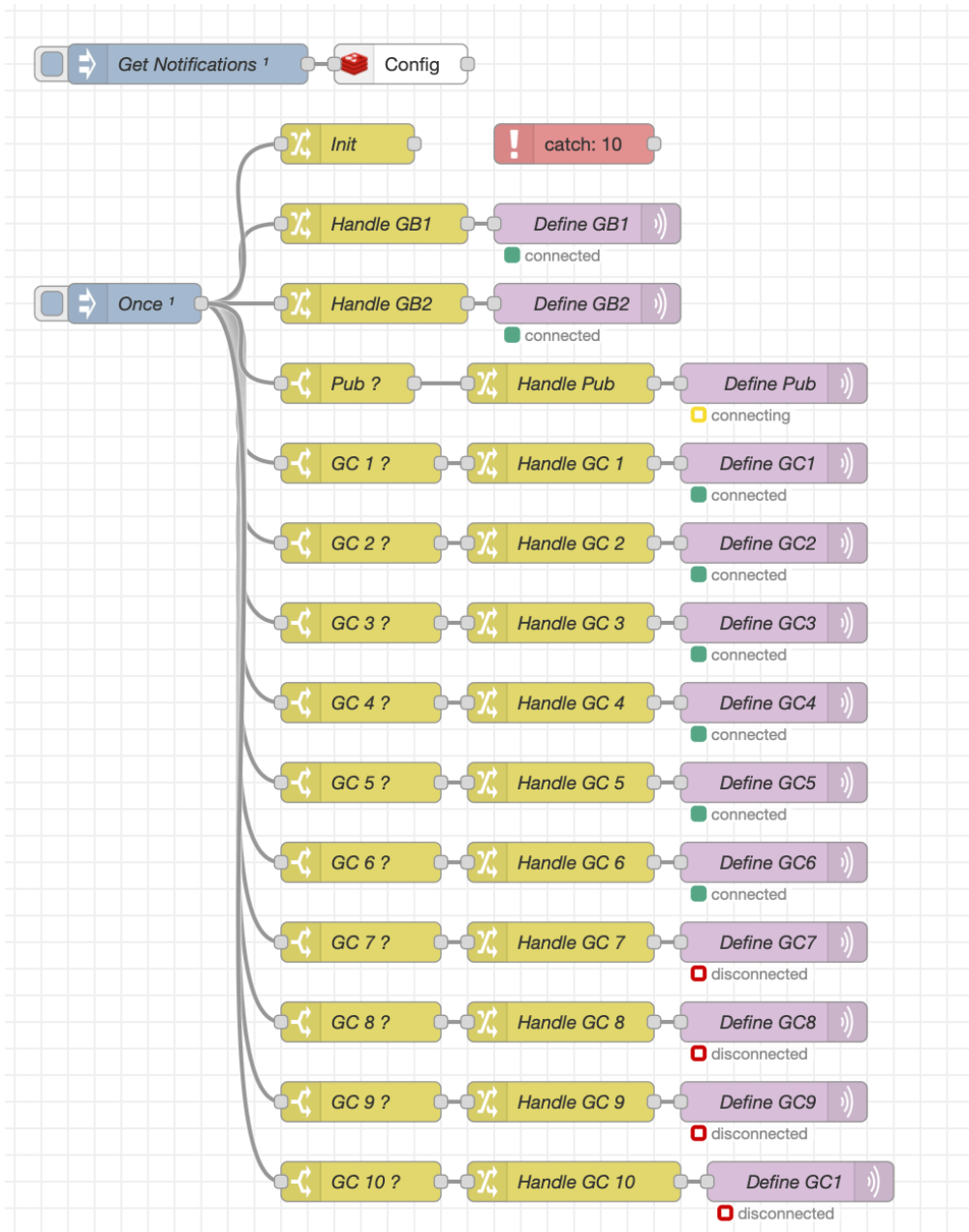
Overview

The SCGC is a Node-RED application that monitors WIS2 Global Cache performance by measuring how quickly each of 10 global caches republishes data that was originally published by origin WIS2 Nodes. It has one main tab, one reusable "Monitor" subflow (instantiated 10 times — once per cache), and uses Redis for correlation and Prometheus for metrics.

Key Design Decisions

- **Correlation key** = data_id + pubtime (not message id) — because two messages about the same data version have the same data_id+pubtime
- **900s TTL on origin messages** — if no cache picks up a message within 15 minutes, it's counted as missed
- **10-second offset** subtracted from delay — accounts for propagation/processing overhead
- **Broker config is environment-driven** — credentials for each GC loaded from env vars, making it portable across deployments
- **Redis key expiry events** — the app subscribes to __keyevent@0__:expired to detect when origin messages expire without being cached (triggering "missed" accounting)





Get Notifications: inject once after 0.1 seconds, then none
 Once: inject once after 0.5 seconds, then none

Message Reception

12 MQTT inputs: 2 origins (GB1, GB2) + 10 global caches (GC1–GC10)

Each input is throttled to **1 msg/second** and deduplicated with an RBE filter

Non-cacheable messages (where `properties.cache == false`) are filtered out

RBE (Report By Exception) Filter

RBE is a Node-RED built-in node that only passes a message through if its payload has **changed** since the last message on the same topic. Identical consecutive messages are silently dropped.

Both RBE nodes (one per origin broker) are configured identically:

Mode: standard rbe — blocks exact duplicates, no deadband or threshold

Property compared: payload (the full message payload)

septopics: true — each MQTT topic is tracked independently, so a repeated payload on topic A does not suppress a new message on topic B

Since WIS2 messages have a unique id, in normal operation every message has a different payload and the RBE passes everything through. Its purpose is to guard against **exact duplicate delivery at the MQTT protocol level** — e.g. a broker retransmitting a message due to a QoS retry or reconnect — which would otherwise be counted twice in the metrics.

Placement in the flow:

GB1: MQTT in → RBE → "Not Cacheable?" switch

GB2: MQTT in → 2-second delay → RBE → "Not Cacheable?" switch

GB2 has an extra 2-second delay upstream of its RBE, likely to stagger processing between the two origin sources and avoid both flooding downstream nodes simultaneously.

Limitations:

Compares raw bytes — messages that are semantically identical but differ in whitespace or field ordering will both pass through

State is held in memory, so after a Node-RED restart the first message on each topic always passes through regardless of whether it is a true duplicate

Origin Message Handling

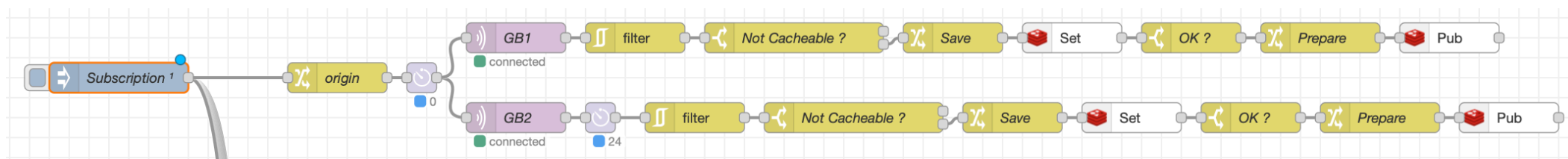
When an origin message arrives:

It's stored in Redis as `wnmid#<id>` with a **900-second TTL**

A Prometheus counter `wmo_wis2_scgc_messages_published_total` is incremented (labelled by `centre_id`)

The message is published to a Redis Pub/Sub channel (`origin/*`) so all Monitor subflows can receive it

Main flow: creating flows for incoming messages from each Global Broker



Subscription: inject once after 10-seconds, then none

Redis Pub/Sub — Origin Message Fan-out

Redis Pub/Sub is used to distribute each origin message to all 10 Monitor subflows simultaneously, without polling.

How it works:

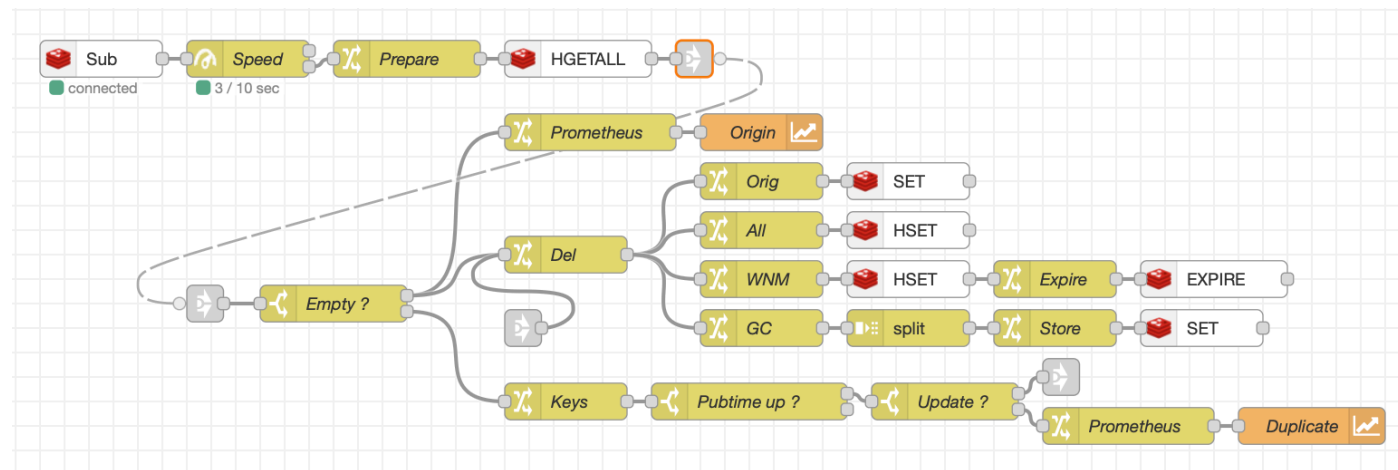
An origin message arrives via MQTT → stored in Redis → then PUBLISH `origin/<centre_id> <payload>`

All 10 Monitor subflow instances are subscribed to `origin/*`
* via PSUBSCRIBE (wildcard)

Each Monitor receives the origin message and holds the metadata it needs to calculate delay when the corresponding cache message arrives

This avoids each Monitor independently querying Redis for origin messages, reducing read load and latency.

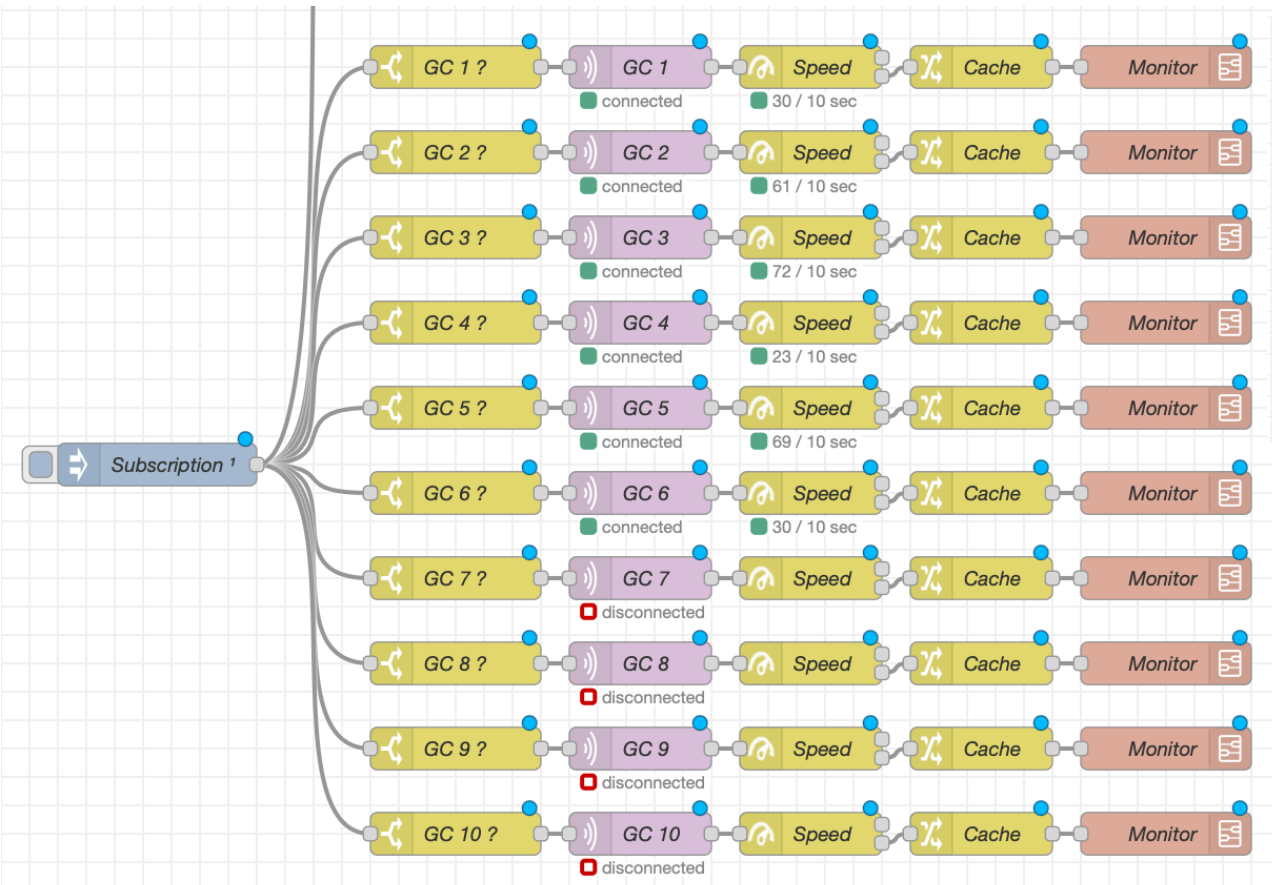
Important caveat: Redis Pub/Sub is fire-and-forget with no persistence. If a Monitor subflow is disconnected or restarting when an origin message is published, it will miss that message entirely — and the corresponding cache message will be counted as missed.



Sub: method PSUBSCRIBE

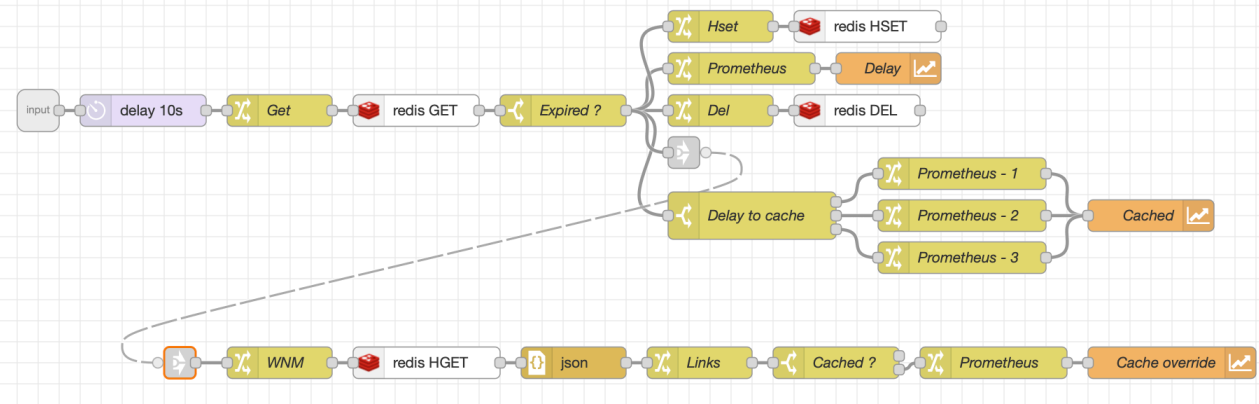
Cache Message Handling — the Monitor Subflow

Main flow: creating flows for each Global Cache based on incoming messages from GC brokers



Subscription: inject once after 10-seconds, then none

Monitor sub flow: handling each message from a GC broker



(looks like this sub flow also identifies where the Global Cache has not cached the data - based on the links in the WNM)

Each global cache has its own Monitor subflow instance. When a cache message arrives:

1. The Monitor matches it to the origin message received via Pub/Sub (matched by properties.data_id + properties.pubtime)
2. If matched: the delay is calculated and categorised into threshold buckets (see below)
3. If not matched within the TTL: counted as **missed**

Delay Calculation

By the time a cache message reaches the delay calculation step, the origin message's pubtime has been retrieved and placed into the message as a millisecond Unix timestamp. The delay is then computed as:

```
delay (seconds) = ( current_time_ms - pubtime_ms ) / 1000 - 10
```

The 10-second subtraction is a fixed offset to account for processing overhead.

Threshold Routing

The delay value is compared against the flow variables set from environment variables, using first-match-wins:

Condition	Bucket	Prometheus delay_threshold label
delay <= CACHE_DELAY_LOW	Fast	low
delay <= CACHE_DELAY_MED	Medium	medium
anything else	Slow	high

Important Subtlety

The delay is measured from pubtime — when the data was originally published by the WIS2 Node — not from when SCGC first saw the origin message on its MQTT subscription. This means the measurement includes any transit time before SCGC received the origin message. It represents **end-to-end latency from publication to global cache availability**, not just the cache's own processing time.

Prometheus Metrics

Metric	Type	Labels	What it measures
wmo_wis2_scg_cache_delay_seconds	gauge	global_cache, centre_id, report_by	Actual delay for each cached message
wmo_wis2_scg_messages_cached_delay_total	counter	global_cache, delay_threshold, centre_id, report_by	Messages per delay threshold bucket
wmo_wis2_scg_messages_missed_total	counter	global_cache, centre_id, report_by	Origin messages NOT seen by a specific cache
wmo_wis2_scg_messages_missed_all_total	counter	centre_id, report_by	Origin messages not seen by ANY cache
wmo_wis2_scg_messages_published_total	counter	centre_id, report_by	Total origin messages seen
wmo_wis2_scg_messages_duplicates_total	counter	centre_id, report_by	Messages with same data_id/ pubtime without an update relation
wmo_wis2_scg_messages_cache_override_total	counter	global_cache, centre_id, report_by	Non-cacheable messages that a cache published anyway

2/ Sensor Centre for Global Cache

2.D/ Tutorials



WORLD
METEOROLOGICAL
ORGANIZATION



3/ **Global Replay (GRep) service**

3.A/ Tech Regs: Guide to WIS, Volume II (updated for INFCOM-4)



WORLD
METEOROLOGICAL
ORGANIZATION



3/ Global Replay (GRep) Service

3.B/ Tutorials



WORLD
METEOROLOGICAL
ORGANIZATION



Thank you!